# README

# digital.vasic.challenges

A generic, reusable Go module for defining, registering, executing, and reporting on challenges (structured test scenarios). Features a plugin-based architecture with built-in assertion evaluation, multi-format reporting, and live monitoring.

## Installation

```
go get digital.vasic.challenges
```

## Quick Start

```
package main

import (
    "context"
    "fmt"
    "log"

    "digital.vasic.challenges/pkg/assertion"
    "digital.vasic.challenges/pkg/challenge"
    "digital.vasic.challenges/pkg/registry"
    "digital.vasic.challenges/pkg/runner"
)

// Define a custom challenge
type HealthChallenge struct {
    challenge.BaseChallenge
}

func NewHealthChallenge() *HealthChallenge {
    return &HealthChallenge{
        BaseChallenge: *challenge.NewBaseChallenge(
            "health_check", "Health Check",
            "Verify all services are healthy", "core",
            nil,
        ),
    }
```

```
}

func (c *HealthChallenge) Execute(ctx context.Context)
        (*challenge.Result, error) {
    result := c.CreateResult()
    result.Status = challenge.StatusPassed
    result.Assertions = []challenge.AssertionResult{
        {Type: "not_empty", Target: "response", Passed: true,
            Message: "Service responded"},
    }
    return result, nil
}

func main() {
    ctx := context.Background()

    // Register challenge
    reg := registry.NewRegistry()
    reg.Register(NewHealthChallenge())

    // Run all challenges
    r := runner.NewRunner(
        runner.WithRegistry(reg),
        runner.WithTimeout(5 * time.Minute),
    )

    results, err := r.RunAll(ctx, &challenge.Config{
        Verbose: true,
    })
    if err != nil {
        log.Fatal(err)
    }

    for _, res := range results {
        fmt.Printf("%s: %s (%v)\n",
            res.ChallengeName, res.Status, res.Duration)
    }
}
```

# Features

- **Challenge framework**: Define, register, and execute structured test scenarios
- **Dependency ordering**: Automatic topological sort (Kahn's algorithm)
- **Assertion engine**: 16 built-in evaluators + custom evaluator support
- **Multi-format reports**: Markdown, JSON, HTML
- **Shell adapter**: Wrap existing bash scripts as challenges
- **Plugin system**: Extend with custom challenge types and assertions
- **Live monitoring**: WebSocket-based real-time dashboard
- **Prometheus metrics**: Built-in challenge metrics
- **Environment management**: Secure env var handling with redaction
- **Challenge banks**: Load definitions from JSON/YAML files
- **Parallel execution**: Run independent challenges concurrently

- **Infrastructure bridge**: Integrates with `digital.vasic.containers`
- **User flow automation**: Multi-platform testing across browser, mobile, API, gRPC, and WebSocket

# User Flow Automation (`pkg/userflow`)

Multi-platform user flow automation framework with adapter-per-platform pattern.

## Adapters (8 interfaces, 21 implementations)

| Interface | Adapters | Technology |
|---|---|---|
| `BrowserAdapter` | PlaywrightCLI, PlaywrightHTTP, Selenium, Cypress, Puppeteer | CDP, W3C WebDriver, CLI |
| `MobileAdapter` | ADB, Appium, Maestro, Espresso | ADB, Appium 2.0, YAML flows, Gradle |
| `DesktopAdapter` | TauriCLI | Tauri WebDriver |
| `APIAdapter` | HTTPAPIAdapter | REST via `pkg/httpclient` |
| `GRPCAdapter` | GRPCCLIAdapter | grpcurl (unary + streaming) |
| `WebSocketFlowAdapter` | GorillaWebSocket | gorilla/websocket (thread-safe) |
| `BuildAdapter` | Gradle, Cargo, NPM, Robolectric | Build tool integration |
| `RecorderAdapter` | PanopticRecorder, ADBRecorder | CDP screencast, ADB |

## Challenge Templates (19 types)

`APIFlowChallenge`, `BrowserFlowChallenge`, `MobileFlowChallenge`, `DesktopFlowChallenge`, `GRPCFlowChallenge`, `WebSocketFlowChallenge`, `BuildChallenge`, `TestRunnerChallenge`, `LintChallenge`, `MultiPlatformChallenge`, plus Recorded variants with video verification.

## Evaluators (12 userflow-specific)

`http_status_ok`, `http_status_created`, `http_status_unauthorized`, `http_json_valid`, `browser_element_visible`, `browser_url_matches`, `mobile_activity_visible`, `mobile_element_exists`, `build_success`, `test_pass_rate`, and more.

See `docs/userflow/` for full adapter documentation and framework comparison.

# Architecture

```
runner.Runner
├── registry.Registry           (Challenge registration +
ordering)
├── assertion.Engine            (16 built-in evaluators)
├── report.Reporter             (Markdown/JSON/HTML)
├── logging.Logger              (Structured logging)
├── monitor.EventCollector      (Live monitoring)
└── plugin.PluginRegistry       (Extensibility)

challenge.Challenge (interface)
├── challenge.BaseChallenge     (Template method base)
├── challenge.ShellChallenge    (Bash script wrapper)
├── userflow.APIFlowChallenge   (HTTP API flow testing)
├── userflow.GRPCFlowChallenge  (gRPC service testing)
├── userflow.WebSocketFlowChallenge (WebSocket flow testing)
└── [your custom challenges]

infra.InfraProvider
└── ContainersAdapter           (Bridge to
digital.vasic.containers)
```

# Built-in Assertion Evaluators

| Evaluator | Description |
|---|---|
| `not_empty` | Value is non-nil and non-empty |
| `not_mock` | Response is not mocked/placeholder |
| `contains` | String contains substring (case-insensitive) |
| `contains_any` | String contains any of the given values |
| `min_length` | String length meets minimum |
| `quality_score` | Numeric score meets threshold |
| `reasoning_present` | Response contains reasoning indicators |
| `code_valid` | Response contains valid code patterns |
| `min_count` | Count meets minimum |
| `exact_count` | Count matches exactly |
| `max_latency` | Response time within limit |
| `all_valid` | All array items are valid |
| `no_duplicates` | No duplicate items in array |
| `all_pass` | All sub-assertions pass |
| `no_mock_responses` | No mocked responses in array |
| `min_score` | Numeric minimum score |

# Anti-bluff guarantees (round-304 — meta Challenge-of-Challenges)

This repository is the cross-cutting **Challenge bank** consumed by every HelixCode-family consumer (Panoptic, security, helix_qa, helix_llm, the core HelixCode app). Because it is itself a test-infrastructure submodule, its own anti-bluff posture must be **meta** — the bank validates the banks. Round-304 (2026-05-19) landed a describe-Challenge meta-runner + inventory ledger that close the recursion.

**Verified surfaces (per `docs/test-coverage.md`):**

- **Bank inventories** — `banks/examples/` (28 generic example banks), `banks/yole/` (7 feature-coverage YAML banks + fixtures), `challenges/` (16 baseline shell-script challenges in `challenges/scripts/`, 1 baseline reference text). Every bank is asserted present + readable + parseable by the describe-runner.
- **Runner-per-bank** — every shell-script bank under `challenges/scripts/` is asserted executable (`+x` bit set) and responds to `--help` or returns a recognisable signature. Missing exec-bit or missing runner = describe-runner FAIL.
- **Paired-mutation per bank** — the describe-runner itself is paired (per CONST-035 / §1.1): `--anti-bluff-mutate` plants a deliberate inventory mismatch (renames a tracked bank file in a tmp tree) and asserts the gate FAILS with exit 99. A meta-runner that PASSes its own mutation is itself a bluff.
- **5-locale fixture** — `challenges/fixtures/payloads.json` exercises the JSON/YAML bank-loader (`pkg/bank`) and assertion engine through every locale (en, de, es, ja, sr — Latin + German + Spanish + CJK + Cyrillic) so non-ASCII bytes survive the load → execute → report pipeline.

**How to invoke:**

```
bash challenges_describe_challenge.sh                    # clean
        PASS (exit 0)
bash challenges_describe_challenge.sh --anti-bluff-mutate #
        planted-mutation FAIL (exit 99)
```

A clean tree MUST yield exit 0; the mutation MUST yield exit 99. Any other outcome is a release blocker per CONST-035 / Article XI §11.9.

**Verbatim 2026-05-19 operator mandate (cascaded per CONST-049 §11.4.17):**

> "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

# License

MIT